DS5003A – Data Engineering

Assignment–3

Date: 14-August-2026

Timing: 2:00 PM to 4:45 PM

Question 1: Find the Closest Value

Write a Python program that accepts an **unsorted list of integers** and a target value. Find the element in the list whose value is closest to the target. Also display the index of that element.

If two elements are equally close to the target, choose the smaller element.

Example:

```
Input:
List = [48, 12, 56, 25, 37]
Target = 40

Output:
Closest value: 37
Index: 4
```

Constraint: Do not use `min()` with a `key` argument.

Question 2: First and Last Occurrence Using Binary Search

Given a sorted list of integers that may contain duplicate values, write a Python program to find the **first occurrence** and **last occurrence** of a given target using Binary Search.

If the target is not present in the list, display `"Element not found"`.

Example:

```
Input:
List = [2, 4, 4, 4, 7, 9, 9, 12]
Target = 4

Output:
First occurrence: 1
Last occurrence: 3
```

Constraint: The search must be performed using Binary Search.

Question 3: Search in a Rotated Sorted Array

Given an integer array `nums` that was originally sorted in ascending order and contains **distinct values**, the array has been rotated at an unknown pivot position.

Write a Python program to find the index at which a given target value `k` is present.

If `k` is not present in the array, return `-1`.

Example 1:

```
Input:
nums = [4, 5, 6, 7, 0, 1, 2]
k = 0

Output:
4
```

Example 2:

```
Input:
nums = [4, 5, 6, 7, 0, 1, 2]
k = 3

Output:
-1
```

Constraints:

- The solution must have a time complexity of $\mathbf{O}(\log \mathbf{n})$.
- Do not create a new array or copy the elements into another array.
- Perform the search directly on the given array.

Question 4: Sort According to Distance from a Given Value

Write a Python program to arrange the elements of a list in ascending order of their **absolute distance** from a given value `x`.

The distance of an element `a` from `x` is given by:

$$|a - x|$$

If two elements have the same distance from `x`, preserve their original relative order.

Example:

```
Input:
List = [10, 5, 3, 9, 2]
x = 7

Distances:
10 -> 3
5  -> 2
3  -> 4
9  -> 2
2  -> 5

Output:
[5, 9, 10, 3, 2]
```

Question 5: Special Even-Odd Sorting

Given a list of integers, rearrange the elements according to the following rules:

- All even numbers must appear first in **ascending order**.
- All odd numbers must appear afterwards in **descending order**.

Example:

```
Input:
[7, 2, 9, 4, 1, 8, 3, 6]

Output:
[2, 4, 6, 8, 9, 7, 3, 1]
```

Constraint: Do not use the built-in `sort()` or `sorted()` functions.

Question 6: Merge Two Sorted Lists

Two lists are already sorted in ascending order. Write a Python program to merge them into a single sorted list.

Example:

```
Input:
List 1 = [2, 6, 9, 15, 21]
List 2 = [1, 4, 10, 18, 25]

Output:
[1, 2, 4, 6, 9, 10, 15, 18, 21, 25]
```

Constraints:

- Do not use `sort()` or `sorted()`.
- Do not concatenate the two lists and then sort the result.
- Construct the final list by comparing elements from the two sorted lists.

Question 7: Partition a List Around a Pivot

Given a list of distinct integers and a pivot value that is present in the list, rearrange the elements such that:

- all elements smaller than the pivot appear before the pivot,
- the pivot appears next, and
- all elements greater than the pivot appear after the pivot.

The relative order of the elements within the smaller and greater groups does not matter.

Example:

```
Input:
List = [9, 4, 7, 3, 10, 5]
Pivot = 5

Possible Output:
[4, 3, 5, 9, 7, 10]
```

Constraints:

- Do not use `sort()` or `sorted()`.
- The pivot must appear exactly once in the resulting list.

Question 8: Count the Number of Incorrect Positions

Given an unsorted list of integers, determine the **number of positions in which the elements are not in their correct positions when the elements are arranged in ascending order**.

Example 1:

```
Input:
[1, 3, 2, 4, 5]

Output:
Number of different positions: 2
```

Example 2:

```
Input:
[4, 2, 1, 3]

Output:
Number of different positions: 3
```